

毕业论文（设计）任务书

学生姓名	曹志隆	学号	32301288	专业 班级	软件工程 2304	电话	13868360863
毕业论文（设计）中文题目		面向超级个体的协作开发 workflow 平台设计与开源集成					
毕业论文（设计）外文题目		Design and Open-Source Integration of a Collaborative Development Workflow Platform for Solo Developers					
指导教师 1	侯宏仑	职称	副教授	所在单位	计算机与计算科学学院	电话	13071858629
指导教师 2		职称		所在单位		电话	
选题类型	☒ 工程设计 ☐ 理论研究 ☐ 实验研究 ☐ 产品开发 ☐ 设计创作 ☐ 专题研究 ☐ 应用研究 ☐ 其他						
选题来源	☐ 国家重点研发计划项目 ☐ 国家社科规划、基金项目 ☐ 国家自然科学基金项目 ☐ 中央、国家各部门项目 ☐ 教育部人文、社会科学研究项目 ☐ 省（自治区、直辖市）项目 ☐ 国际合作研究项目 ☐ 港、澳、台合作研究项目 ☐ 企、事业单位委托项目 ☐ 外资项目 ☐ 国防项目 ☒ 学校自选项目 ☐ 非立项 ☐ 其他						
是否在实验、实习、工程实践和社会调查等社会实践中完成						☒ 是 ☐ 否	
毕业论文（设计）时间		【待填：学院规定起止日期】					
阶段工作内容		工作进度安排周次					
查阅文献		第 5 周（第七学期）至第 7 周（第七学期）					
文献综述和外文翻译		第 8 周（第七学期）至第 12 周（第七学期）					
开题报告		第 13 周（第七学期）至第 17 周（第七学期）					
研究、设计、开发		第 1 周（第八学期）至第 7 周（第八学期）					
撰写毕业论文（说明）		第 8 周（第八学期）至第 12 周（第八学期）					
答辩与修改		第 12 周（第八学期）至第 15 周（第八学期）					
“阶段工作内容”包括：查阅文献、调研、撰写综述、翻译外文资料、开题、实验或设计、绘图、撰写毕业论文或设计说明书、定稿及答辩等；							
论文（设计）的主要内容及要求：（以设计作品为主要成果形式的毕业设计要求以各分院实施方案为准）							
1、开题报告和文献阅读							
(1) 文献阅读：查阅文献 15 篇（含）以上，其中外文文献 2 篇（含）以上，近三年公开发表的文献 5 篇（含）以上，书籍不超过 3 本，期刊〔J〕和论文集〔C〕10 篇（含）以上，包括导师指定的全部参考文献。							
(2) 外文翻译：3000 字以上。							

(3) 文献综述：3000 字以上，包括国内外研究现状、研究方向、进展情况、存在问题和参考依据等。

(4) 开题报告：3000 字以上，包括选题的意义、可行性分析、研究内容、研究方法、拟解决的关键问题、预期结果和研究进度计划等。

2、论文：10000 字以上，包括绪论、正文、结论、参考文献等。法学、人文、经管类专业 8000 字以上，特殊专业应满足分院根据培养目标另行制定的相应要求。

3、课题要解决的主要问题和具体要求（填写如下）：

一、课题定位与目标

本课题选择独立开发者/小型开发团队维护软件项目作为具体业务场景，设计并实现一个人工智能生成内容（Artificial Intelligence Generated Content, AIGC）协作开发工作流平台。平台以项目目标、需求资料、代码仓库、决策记录和验收标准为事实输入，通过模型上下文协议（Model Context Protocol, MCP）连接代码智能体、Git/GitHub 及测试工具，把需求澄清、任务规划、代码执行、人工审查、测试和交付组织为可编辑、可执行、可追溯的工程闭环。

毕业设计以可运行平台原型和工程验证结果为主要成果。评价维度按照“与课题目标直接对应、相互边界清晰、能够取得可验证证据”三项原则确定。依据软件产品质量、人机协作、AI 风险治理和平台工程研究，选择五个核心维度：（1）任务有效性，考察验收条件、需求覆盖和范围偏离；（2）工程产物质量，考察测试、缺陷返工、可维护性和安全问题；（3）人机协作与可控性，考察状态理解、澄清、审批、审查、撤销和人工接管；（4）过程可信与可追溯性，考察证据链、权限审批、复现恢复和敏感信息保护；（5）平台工程能力，考察可靠性、MCP 互操作、Runner 可替换性、工具集成和部署能力。五个维度分别回答任务是否做对、产物是否合格、用户能否控制 AI、过程能否审计追溯以及平台能否可靠运行，并可由需求追踪、测试报告、交互事件、Trace/Asset 和部署验证提供证据。

同时选择不将研发效能与资源效率列为核心维度，是因为真实 AI 开发任务难以等价，模型版本、随机输出和多智能体并行又使耗时、Token、重试与费用难以公平比较和归因；相关数据仅用于案例复盘和运行诊断，不据此宣称平台具有普遍的效率或成本优势。

二、主要交付内容

（1）需求规格与用户体验（User Experience, UE）核心原型。依据项目现有核心想法、开发计划和 Focus OS 界面规范，形成独立开发者/小型团队的典型用户旅程、功能范围、异常场景和验收标准；完成项目创建或导入、Project Brief 确认、工作流成果视图与完整流程、Task 工作区、Assist 协作、审批、Diff Review、资产和交付结果等核心界面原型。

（2）工作流与项目上下文模块。采用“Project—Workflow—Workstream—Task”层级，把 Project Brief 中的功能、里程碑、风险和验收条件映射到可独立验收的任务；用 Node Contract 声明任务目标、输入、输出、允许工具和验收标准。AIGC 生成候选工作流，确定性校验器检查依赖、需求覆盖和资产流，用户确认后形成正式版本；任务执行前由系统从显式输入、直接依赖、项目决策和仓库快照构建结构化上下文包（Context Pack）。

（3）MCP 工具集成与受控执行模块。系统通过 MCP 能力目录和 Runner Adapter 接入 Codex 等代码智能体，页面、REST、MCP 和终端入口复用同一业务服务、权限与审计规则。任务在受管 Repository Workspace 及隔离 worktree 中执行，记录命令、文件变更、工具调用和状态事件；写入、敏感操作和外部交付经过人工确认、checkpoint 与 Diff Review，凭据不进入 Context Pack、Trace 或前端响应。

（4）资产证据与工程交付模块。将需求、决策、代码变更、测试报告和交付记录保存为带版本和来源关系的 Asset，使用 Trace、Workspace Digest、内容哈希及证明记录关联任务输入、执行过程与输出。验收通过后形成 Git 提交或 Draft Pull Request，并以需求追踪、工程产物、人工决策、过程证据和平台测试数据支撑多维评价；任务耗时、模型用量、工具调用和重试只作为具体运行的描述与诊断信息。

三、基本技术路线

系统采用“AIGC 编排层+确定性控制层”的双层技术架构：AIGC 负责根据目标动态规划任务、选择工具、观察结果并决定是否修正或重规划；Node.js 控制层负责权限、状态持久化、结构校验、审批和安全边界。

（1）目标与上下文建模：将用户目标、Project Brief、需求资料、代码仓库快照、约束条件和验收标准组织为结构化输入，并建立 MCP 能力目录，使 AI 能够同时理解“要完成什么、当前项目是什么状态、可以调用哪些能力”。

（2）动态工作流生成：由 LLM Orchestrator 依据目标和上下文执行 Plan-and-Execute 规划，动态生成 Workstream/Task 组成的 DAG、依赖关系、输入输出和验收标准，而不是预置固定节点。确定性校验器检查 DAG 无环、需求覆盖、依赖完整和能力可用性，用户确认后形成正式工作流。

（3）智能体任务与 MCP 编排：每个 Task 作为具有目标、上下文、可用工具和完成条件的智能体任务。AIGC 根据当前计划和执行观察自主选择 MCP 工具、代码执行器及调用顺序；Node.js 后端提供统一能力入口和执行约束，不硬编码每类任务的业务路径。

（4）动态 Context Pack 与记忆组织：执行前按 Task 从显式输入、直接依赖、项目决策、相关代码和仓库快照中检索、压缩并组装最小充分 Context Pack；上下文随任务和版本动态更新，避免把全部历史对话直接塞给模型，并降低信息遗漏、冲突和上下文漂移。

（5）执行、反思与自我修正：采用“Plan—Execute—Observe—Reflect”循环，智能体执行工具或修改代码后读取测试、命令和错误结果，在限定次数内反思原因并修正；当目标、依赖或验收条件发生变化时生成重规划提案，超过重试上限、置信度不足或涉及高风险操作时转交人工处理。

（6）人机协同与确定性治理：用户负责确认目标、正式工作流、高风险写入和最终交付，并通过审批、Diff Review、撤销和人工接管保持控制。受管 worktree、权限校验、凭据隔离、幂等和失败恢复约束智能体行为；这些机制提供安全边界，但不替代 AIGC 的动态规划与工具决策。

（7）分层测试与 AI Eval：对权限、状态、接口、数据和安全边界继续执行单元、集成、端到端、故障和安全测试；对动态工作流、工具选择、上下文构造、反思和重规划建立版本化 Eval 场景集。在固定 Brief、仓库快照、模型、Prompt 和能力集合下重复运行，评价需求覆盖、DAG 有效性与可执行性、工具选择正确性、验收通过、范围偏离及自我修正结果；Prompt、模型或编排策略变化后执行回归 Eval 并报告波动和局限。

（8）集成部署与迭代优化：集成 Codex、MCP、Git/GitHub 和 Docker，完成代表性软件任务及异常场景验证；依据 Eval、自动化测试和人工审查结果迭代 Prompt 模板、规划器、Context Pack 选择策略、工具权限、停止条件和交互方式，最终完成可运行原型、部署文档、测试与评测报告。

四、核心工程问题与处理重点

（1）需求与 UE 核心原型的范围一致性。Project Brief 作为需求基线，功能、验收条件、里程碑和风险均需映射到负责的 Workstream/Task；界面同时呈现目标、依据、执行状态、阻塞原因和人工决策入口，使用户能够理解并控制 AIGC 工作过程。

（2）架构设计中的上下文主权与组件可替换性。项目上下文、资产、决策和 Trace 由平台持有，代码智能体只消费按任务组装的 Context Pack；Runner Adapter 和 MCP 能力目录隔离具体模型及工具差异，REST、MCP 和网页入口统一调用领域服务。

（3）实现集成中的执行边界与状态一致性。外部代码源保持只读，写操作限定在受管 checkout/worktree；权限、路径、敏感信息、写锁、幂等、checkpoint、失败恢复和仓库快照共同约束长任务执行，避免并发覆盖、重复交付和越权访问。

（4）测试评价中的证据充分性与结论边界。采用标准化系统测试和代表性场景案例，分别验证功能验收、产物质量、人工控制、证据追溯和平台可靠性。案例可记录耗时、Token、重试和人工决策，用于还原执行过程和识别运行开销。

五、开发过程与进度要求

- (1) 需求与 UE 原型阶段：以核心想法、愿景范围、开发计划和现有界面规范为输入，完成场景边界、用户旅程、需求规格、非功能要求、信息架构和可交互原型，并建立需求到验收标准的追踪表。
- (2) 架构与最小闭环阶段：完成领域模型、模块边界、接口、权限和数据方案，优先跑通“Project Brief—Workflow—Task—Context Pack—受控执行—Diff Review—测试—交付”的最小闭环。
- (3) 实现集成与质量加固阶段：分阶段接入 Codex Runner、MCP、受管仓库、资产证据和 Git/GitHub 交付；每个增量配置单元、集成和端到端测试，并覆盖路径隔离、敏感信息、幂等、审批、回滚和异常恢复。
- (4) 场景验证与成果交付阶段：选择具有代表性的功能增量和异常处理场景，验证平台闭环及五维评价要求；根据测试、可用性与案例证据优化工作流、上下文和交互，运行时间与模型用量只作案例说明，最终完成部署包、源代码、项目文档、测试报告、场景验证报告和答辩材料。

六、预期成果与验收要求

- (1) 提交可运行的平台原型及源代码，完整演示项目接入、Brief 确认、工作流生成与校验、Task 上下文构建、受控执行、人工审查、自动化测试、资产绑定和 Git 交付。
- (2) 提交需求规格、UE 核心原型、系统架构与数据模型、接口说明、部署运行手册、测试用例、需求追踪矩阵和场景验证报告，关键结论可追溯到代码、测试、Trace、Asset 或版本库记录。
- (3) 系统通过静态检查、单元测试、集成测试和端到端测试，重点验证需求覆盖、上下文范围、权限路径、敏感信息、审批幂等、失败恢复、输出绑定和交付一致性；未执行的真实外部服务测试明确标记。
- (4) 至少完成两类代表性软件开发场景验证，分别覆盖功能增量和带异常或安全约束的任务；每个场景保留需求、上下文、执行、审批、Diff、测试和交付证据，按任务有效性、工程产物质量、人机协作与可控性、过程可信与可追溯性、平台工程能力报告结果及局限。涉及时间、Token 和费用的数据注明任务、模型、版本、上下文与环境，仅作为描述性证据。
- (5) 平台形成需求、决策、执行、测试和交付的证据链，正式 Task 的必需输出全部绑定并通过验收后方可完成，代码交付包含 Diff、测试结果、提交或 Draft PR 记录。
- (6) 完成不少于 10000 字的毕业论文（设计说明书），包括绪论、相关工作、需求与 UE 设计、系统架构、实现、测试与工程评价、结论与展望，并按 GB/T 7714—2015 著录参考文献。

推荐参考文献：

[1] WANG Y, ZHONG W, HUANG Y, et al. Agents in software engineering: Survey, landscape, and vision[J]. Automated Software Engineering, 2025, 32(2): 70. DOI:10.1007/s10515-025-00544-2.

[2] QIAN C, LIU W, LIU H, et al. ChatDev: Communicative agents for software development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. Bangkok: Association for Computational Linguistics, 2024: 15174-15186. DOI:10.18653/v1/2024.acl-long.810.

[3] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37. [S.l.]: Neural Information Processing Systems Foundation, 2024: 50528-50652. DOI:10.52202/079017-1601.

[4] WANG L, MA C, FENG X, et al. A survey on large language model based autonomous agents[J]. Frontiers of Computer Science, 2024, 18(6): 186345. DOI:10.1007/s11704-024-40231-1.

[5] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: Meta programming for a multi-agent collaborative framework[C/OL]//The Twelfth International Conference on Learning Representations. [S.l.]: OpenReview, 2024[2026-07-24]. <https://openreview.net/forum?id=VtmBAGCN7o>.

[6] MODEL CONTEXT PROTOCOL. Specification: 2025-06-18[EB/OL]. (2025-06-18)[2026-07-24]. <https://modelcontextprotocol.io/specification/2025-06-18>.

[7] BARKE S, JAMES M B, POLIKARPOVA N. Grounded Copilot: How programmers interact with code-generating models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85-111. DOI:10.1145/3586030.

[8] AMERSHI S, WELD D, VORVOREANU M, et al. Guidelines for human-AI interaction[C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. New York: ACM, 2019: 1-13. DOI:10.1145/3290605.3300233.

- [9] PERRY N, SRIVASTAVA M, KUMAR D, et al. Do users write more insecure code with AI assistants?[C]//Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. New York: ACM, 2023: 2785-2799. DOI:10.1145/3576915.3623157.
- [10] VAN DER AALST W M P, TER HOFSTEDE A H M, KIEPUSZEWSKI B, et al. Workflow patterns[J]. Distributed and Parallel Databases, 2003, 14(1): 5-51. DOI:10.1023/A:1022883727209.
- [11] HAUGE O, AYALA C, CONRADI R. Adoption of open source software in software-intensive organizations: A systematic literature review[J]. Information and Software Technology, 2010, 52(11): 1133-1154. DOI:10.1016/j.infsof.2010.05.008.
- [12] FRANCO-BEDOYA O, AMELLER D, COSTAL D, et al. Open source software ecosystems: A systematic mapping[J]. Information and Software Technology, 2017, 91: 160-185. DOI:10.1016/j.infsof.2017.07.007.
- [13] MOREAU L, CLIFFORD B, FREIRE J, et al. The Open Provenance Model core specification (v1.1)[J]. Future Generation Computer Systems, 2011, 27(6): 743-756. DOI:10.1016/j.future.2010.07.005.
- [14] TABASSI E. Artificial Intelligence Risk Management Framework (AI RMF 1.0)[R]. Gaithersburg, MD: National Institute of Standards and Technology, 2023. DOI:10.6028/NIST.AI.100-1.

指导教师签名: _____

年 月 日

毕业论文（设计）工作指导小组意见:

☐ 同意下达任务书 ☐ 不同意下达任务书

负责人签名: _____

年 月 日